

ScrollSense - Assignment 1

Mithilesh-da24b048

September 13, 2026

1 Deliverable A - Assumptions and Scope

1.1 A.1 Twelve Assumptions

Going through section 2 of the client brief, twelve places stood out where two different readings would actually change the shape of a table, not just a data type or a label. Each one below states the actual question, the assumption taken, the reasoning behind it, and what would have needed to change in the schema if the opposite reading had been picked instead.

#	Where	Ambiguity / Question	My Assumption	Why	What breaks if opposite
A1	Sec 2.1, Users	Can one account have both a phone number and a Google account, or exactly one registration method?	A user may have multiple authentication identities, including both a phone number and a Google account at the same time	The brief gives alternative registration methods but never says they are mutually exclusive, so nothing forces a choice between them	Authentication would need a separate table holding one row per method per user, instead of two plain nullable columns on AppUser
A2	Sec 2.1, Users	Are previous handles retained as history, or does only the current handle get stored?	Previous handles are retained as history, each one its own row, with the current active handle marked by a null end date	Handles change over time and the brief describes exactly this scenario through the support ticket example, so a single overwritten column would lose that history entirely	A single current handle column cannot answer what a user was called last year, and the temporal Handle table becomes unnecessary if history is not required
A3	Sec 2.1, Users	When a weekly refresh writes a new inferred interest score, does suppression on that category carry forward automatically, or does it reset?	Suppression is a standing preference tied to the user and category pair, and it binds every future inferred score in that category regardless of the weekly refresh	The brief is explicit that suppression has to survive refreshes, and the only way that is guaranteed is if suppression lives outside the row that gets overwritten each week	If suppression lived inside the same row as the score, every refresh job would need to remember to copy it forward, and a single missed copy would silently unsuppress a category the user had turned off
A4	Sec 2.3, Social Graph	If a follow ends and later starts again between the same two users, is that one relationship or two separate periods?	Each continuous follow period is stored as its own row, so ending a follow and starting a new one later creates a second row rather than reopening the old one	This preserves the social graph as it looked at different points in time, which the brief needs for the growth team's historical queries	A single mutable follow row could not reconstruct what the graph looked like before it was broken, since there would be no record that the first follow ever ended
A5	Sec 2.4, Watch Telemetry	Can the same explicit signal type be logged more than once for the same user and video?	Signals are stored at event grain, so the same user can generate more than one row of the same signal type against the same video over time	Real behaviour like sharing a clip twice or commenting more than once is normal, and the event grain preserves that instead of collapsing it	A uniqueness constraint on user, video, and signal type would incorrectly block legitimate repeated actions such as a second share or a second comment

#	Where	Ambiguity / Question	My Assumption	Why	What breaks if opposite
A6	Sec 2.4, Watch Telemetry	Can one Share event have more than one destination, or exactly one?	Each Share row has exactly one destination. Sharing to two destinations produces two separate Share rows	This keeps the event grain of Share clean and destination as a single valued attribute, matching how the brief lists destinations as a fixed small set	If one share could hold several destinations, destination could no longer be a plain column, and a separate junction table would be needed just to hold the destination list per share
A7	Sec 2.4, Watch Telemetry	Is a view a separate stored fact, or a condition derived from view segments?	A view is not a stored row of its own. It is a derived condition computed from ViewSegment rows crossing the completion threshold	The brief defines a view through a watch time and completion threshold rather than describing it as something a user explicitly does, so treating it as a computed condition matches that definition directly	Storing View as its own table would introduce a second copy of information that ViewSegment already holds, and the two could drift out of sync if an update to one were missed
A8	Sec 2.2, Creators and Content	Can a video's ownership change after it is published, or is the owner fixed for the video's lifetime?	A video's owner is fixed once the video is created and never changes afterward	The brief describes an owner as identifying who uploaded a clip but never describes any transfer-of-ownership flow anywhere in the case study	If ownership could change, owner id alone on Video would not be enough, and a temporal ownership history table would be required to answer who owned a clip on a given date
A9	Sec 2.5, Agent Layer	Can a nested tool call have more than one parent, or exactly one?	Each tool call has exactly one parent. Root level calls belong directly to a turn with no parent call at all	This makes tool execution a strict tree rather than a graph, which is what the brief's description of nested calls implies, and it supports recursive nesting to any depth without extra tables	If a call could have multiple parents, a single parent call id column would not be enough, and tool calls would need a many to many join table instead of the simple self reference used here
A10	Sec 2.5, Agent Layer	Is a recommendation identified only by the video it points to, or by its specific shelf occurrence?	Each recommendation is its own shelf occurrence, tied to one turn and one position. A later watch of that same video is linked back to that specific occurrence, not just to the video in general	The same video can be recommended again in a different turn or a different position, so treating occurrence as the real grain avoids collapsing two different recommendation events into one	If video id alone identified a recommendation, there would be no way to tell which specific recommendation a later watch was actually satisfying, and F12 would become impossible to answer correctly

#	Where	Ambiguity / Question	My Assumption	Why	What breaks if opposite
A11	Sec 2.5, Agent Layer	Is a turn's model price fixed at the moment the turn executes, or recomputed later from whatever the current price happens to be?	The price that applies to a turn is the one active at the exact time the turn occurred, looked up through the turn's own timestamp, and it never changes afterward	Finance requires historical costs to stay stable even after a vendor changes prices, and the brief states this rule directly	If price were looked up live at reporting time, every past Finance report would quietly change value the next time a vendor updated their rate, which is exactly the instability the brief warns against
A12	Sec 2.7, Failure Scenarios	Is the current moderation state kept as its own independently updated fact, or is it always derived from the moderation history log?	Moderation history is the single source of truth. Current state is never stored separately, it is always derived by reading the most recent row for a video	The brief describes exactly this kind of drift happening in the founders' original database, where a cached status column and the real history disagreed after a partial write	Keeping a separately maintained current state column risks the two values disagreeing if one write succeeds and the other does not, recreating the same taken down versus live inconsistency the brief warns about

1.2 A.2 What Is Being Descoped

Three things were deliberately left out of the build, even though a looser reading of the brief could suggest otherwise.

The first is the machine learning model that actually decides feed ranking order or produces the confidence scores behind inferred interests. Section 2.4 mentions a model version tag stored on every impression, which on a quick read could look like a request to build the ranking system itself. This is a database design assignment, though, so the schema only needs to record which model version made a decision, not implement the decision-making logic. The model version column on Impression and the score column on InferredInterest already exist specifically so that a real ranking system could start writing into them later without needing any schema change.

The second is working implementations of the agent's own tools, meaning search videos, get user history, and fetch trending audio actually doing something rather than just being named. Section 2.5 describes these tools in enough detail that it can read like a build request, but the actual grading only checks whether the schema can record a call happening, its arguments, its result, how long it took, and how deeply it was nested. None of that requires the tool to genuinely work behind the scenes. Arguments and results are stored as JSON text specifically so a real tool implementation could plug into the same table later.

The third is any kind of automated handle theft detection system. The recurring support ticket described in section 2.1, about two active accounts colliding on a handle that differs only by capitalization, could be read as asking for some kind of detection or monitoring layer. The actual fix already applied, a case insensitive uniqueness rule scoped to active handles only, stops this bug from recurring at the root cause. Building detection on top of a bug that no longer exists does not add real value here. A support ticket table could be layered on top of the existing user and handle history tables later without disturbing anything already built.

1.3 A.3 Revision Log

Stages 1 and 2 were not submitted as separate checkpoints with instructor feedback in between. Deliverables A through D were built and checked together in one pass before the final deadline, so instead of logging changes made in response to feedback, each part was checked directly against its own marking criteria in the task sheet before being finalized.

The twelve assumptions above ended up weighted more toward sections 2.3 through 2.5 rather than being spread evenly across the whole brief. This was intentional. The task sheet's own hint notes that most of the real cardinality

ambiguities live in the social graph, telemetry, and agent sections, and separately warns that a set of assumptions drawn only from sections 2.1 and 2.2 loses marks.

2 Deliverable B - Conceptual Design

Note on the ER Diagram

The entity relationship diagram was built in dbdiagram.io from the entity and relationship inventory above, exported as DIAGRAM.pdf, and added to the project repository alongside the SQL files.

2.1 Step 1: Entities

Entity	One row represents	Identifier	Kind
AppUser	one registered account	user_id	strong
Handle	one handle a user held over some stretch of time	user_id, started_at	temporal
InterestCategory	one category the recommendation system knows about	category_id	strong
InferredInterest	one category and score for a user, this week's value	user_id, category_id	overwritten weekly
InterestSuppression	one category a user chose to suppress, kept forever	user_id, category_id	weak
Creator	the creator role layered on top of a user	user_id	strong, optional, 1:1 with AppUser
CreatorTierPeriod	one stretch of time a creator sat at one tier	creator_id, valid_from	temporal
Video	one uploaded clip	video_id	strong
Hashtag	one distinct lowercased hashtag	hashtag_id	strong
VideoHashtag	one video carrying one hashtag	video_id, hashtag_id	junction
AudioTrack	one track that can be attached to videos	track_id	strong
VideoModerationEvent	one moderation decision at one point in time	event_id	event
Follow	one directional follow, possibly ended	follower_id, followee_id, started_at	temporal
Block	one user blocking another	blocker_id, blocked_id	weak
Mute	one user muting another	muter_id, muted_id	weak
Impression	one clip entering one user's viewport once	impression_id	event
ViewSegment	one continuous stretch of playback inside an impression	segment_id	weak
Like_	one like, possibly retracted later	like_id	temporal
Retraction	one retraction, its own independent event	retraction_id	event
Share	one share to one destination	share_id	event
Comment	one comment on one video	comment_id	strong
SignalEvent	one not interested, report, or follow-from-feed signal	signal_id	event
AgentSession	one conversation between a user and the agent	session_id	strong
Turn	one exchange, user message plus assistant message	turn_id	weak

Entity	One row represents	Identifier	Kind
ToolCall	one tool call, possibly nested inside another	call_id	weak, self referencing
PromptTemplateVersion	one frozen version of a template's text	template_id, version_no	append only
ModelPricePeriod	one price for one model over a stretch of time	model_id, priced_at	temporal
JudgeScore	one automated evaluation of a turn	score_id	weak
UserRating	one thumbs up or down from a user on a turn	rating_id	weak
Recommendation	one clip at one shelf position on a turn's response	turn_id, position	weak
TurnUsage	one turn's token counts for billing	turn_id	weak

Strong means the entity stands on its own. Weak means it only makes sense attached to a parent row. Event means something that happened once and is never edited afterward. Temporal means the entity tracks a value across time. Junction resolves a many to many relationship between two other entities.

2.2 Step 2: Relationships

Relationship	From → To	Cardinality	Participation	Attributes
uploads	AppUser → Video	1 to 0..N	user optional, video mandatory	uploaded_at
uses	Video → AudioTrack	0..N to 0..1	both optional	none
tags	Video → Hashtag	0..N to 0..N	both optional	none
becomes	AppUser → Creator	1 to 0..1	user mandatory	became_creator_at
holds tier	Creator → CreatorTierPeriod	1 to 1..N	both mandatory	valid_from, valid_to
moderated by	Video → VideoModerationEvent	1 to 0..N	video mandatory	decided_at, decided_by
follows	AppUser → AppUser	0..N to 0..N	both optional	started_at, ended_at
blocks	AppUser → AppUser	0..N to 0..N	both optional	blocked_at
mutates	AppUser → AppUser	0..N to 0..N	both optional	muted_at
views	AppUser → Video via Impression	0..N to 0..N	both optional	ts, feed_position, model_version
segments of	Impression → ViewSegment	1 to 0..N	impression mandatory	started_at, ended_at
likes	AppUser → Video	0..N to 0..N	both optional	liked_at, retracted_at
starts	AppUser → AgentSession	1 to 0..N	user mandatory	started_at
contains	AgentSession → Turn	1 to 0..N	session mandatory	turn_index
issues	Turn → ToolCall	1 to 0..N	turn mandatory	called_at
nests	ToolCall → ToolCall	0..1 to 0..N	both optional	none
answered with	Turn → PromptTemplateVersion	0..N to 1	version mandatory	none
priced by	Turn → ModelPricePeriod	0..N to 1	period mandatory	none
recommends	Turn → Video via Recommendation	0..N to 0..N	both optional	position

2.3 Step 4: How Each Temporal Fact Is Stored

Entity	Overwrite	Validity interval	Event log	Why
Creator tier		X		Finance needs to ask what tier a creator held on a past date, and tiers hold steady for a while, so a start and end date fits better than logging every change as its own separate event
Video moderation state			X	every decision needs an actor attached, either classifier or reviewer, and the state can change often, so a log lets current state be derived instead of trusting a column that could silently drift
Inferred interests	X			the brief only requires suppression to survive the weekly refresh, not the score history itself, so overwriting the score is fine once suppression is pulled into its own separate table
Likes and retractions			X	retraction is treated as its own independent signal by the brief, not a simple status flip, so both the like and the retraction stay as permanent rows
Prompt template text			X	old responses must stay linked to the exact text used at the time, which only holds if a version is never edited once written
Model pricing		X		prices hold steady until a vendor changes them, so a validity period lets a turn's own timestamp find the correct historical price

The Three Hardest Design Calls

The first hard call was moderation state. Adding a current state column directly on Video and updating it whenever a new decision comes in would have been the simpler route to write, but the brief describes exactly this setup failing in the founders' own database, where the history write and the status update land as two separate writes that can get split apart if something crashes in between. Because of that, only the event log is kept, and current state is always derived by reading the latest row for a video. This costs an extra query every time someone wants to know the current state, but it also means the two values can never disagree, since there is only ever one source of truth to read from.

The second hard call was whether Creator needed to be a separate table at all. Simply adding a tier column straight onto AppUser would have meant one less join, but tier changes over time and Finance specifically needs to look back at past tiers, and most users are never creators in the first place. Mixing temporal creator specific history into the main user table felt like the wrong trade. Creator ended up as its own table, optional and one to one with AppUser, holding its own tier history in a linked table.

The third hard call was how far to carry the impression, view segment, watch time chain. Treating one impression as roughly equivalent to one view would have meant noticeably less modeling work, since the overwhelming majority of impressions never turn into anything. But the brief is specific that a single impression can produce several distinct viewing segments, and that watch time and completion rate need to remain separate ideas rather than being conflated. Keeping all three levels apart means more joins at query time later, but it directly avoids double counting watch time, which the brief calls out as a common mistake.

3 Deliverable C - Logical Design, Schema, and Relational Algebra

3.1 C.1 Relations

The relations below map directly onto the entities and relationships identified in Deliverable B. The listing shown is the actual working DDL from schema.sql, run and verified against a live SQLite 3.44 database, not a hand typed summary of it.

Listing 1: Pragmas and a quick type affinity probe table

```
1 PRAGMA foreign_keys = ON;
2 PRAGMA journal_mode = WAL;
3
4 CREATE TABLE TypeAffinityTest (
5     id INTEGER PRIMARY KEY,
6     strict_int INTEGER
7 );
```

Listing 2: Users, handles, and inferred interests

```
1 CREATE TABLE AppUser (
2     user_id INTEGER PRIMARY KEY,
3     phone TEXT,
4     google_sub TEXT,
5     display_name TEXT NOT NULL,
6     account_state TEXT NOT NULL CHECK (account_state IN
7     ('active', 'deactivated', 'pending_deletion')),
8     deletion_requested_at TEXT,
9     created_at TEXT NOT NULL,
10    CHECK (phone IS NOT NULL OR google_sub IS NOT NULL)
11 );
12
13 CREATE TABLE Handle (
14     user_id INTEGER NOT NULL REFERENCES AppUser(user_id) ON DELETE CASCADE,
15     handle TEXT NOT NULL,
16     started_at TEXT NOT NULL,
17     ended_at TEXT,
18     PRIMARY KEY (user_id, started_at)
19 );
20
21 CREATE UNIQUE INDEX idx_handle_active_unique
22 ON Handle(handle COLLATE NOCASE)
23 WHERE ended_at IS NULL;
24
25 CREATE TABLE InterestCategory (
26     category_id INTEGER PRIMARY KEY,
27     name TEXT NOT NULL UNIQUE
28 );
29
30 CREATE TABLE InferredInterest (
31     user_id INTEGER NOT NULL REFERENCES AppUser(user_id) ON DELETE CASCADE,
32     category_id INTEGER NOT NULL REFERENCES InterestCategory(category_id)
33     ON DELETE RESTRICT,
34     score REAL NOT NULL CHECK (score BETWEEN 0 AND 1),
35     refreshed_at TEXT NOT NULL,
36     PRIMARY KEY (user_id, category_id)
37 );
38
39 CREATE TABLE InterestSuppression (
40     user_id INTEGER NOT NULL REFERENCES AppUser(user_id) ON DELETE CASCADE,
41     category_id INTEGER NOT NULL REFERENCES InterestCategory(category_id)
42     ON DELETE CASCADE,
43     suppressed_at TEXT NOT NULL,
44     PRIMARY KEY (user_id, category_id)
45 );
```

Listing 3: Creators, tiers, videos, hashtags, and audio

```
1 CREATE TABLE Creator (
2     user_id INTEGER PRIMARY KEY REFERENCES AppUser(user_id) ON DELETE CASCADE,
```

```

3     became_creator_at TEXT NOT NULL
4 );
5
6 CREATE TABLE CreatorTierPeriod (
7     creator_id INTEGER NOT NULL REFERENCES Creator(user_id) ON DELETE CASCADE,
8     valid_from TEXT NOT NULL,
9     valid_to TEXT,
10    tier TEXT NOT NULL CHECK (tier IN
11        ('standard','rising','partner','elite')),
12    PRIMARY KEY (creator_id, valid_from)
13 );
14
15 CREATE TABLE AudioTrack (
16     track_id INTEGER PRIMARY KEY,
17     origin_video_id INTEGER,
18     source TEXT NOT NULL CHECK (source IN ('original','licensed'))
19 );
20
21 CREATE TABLE Video (
22     video_id INTEGER PRIMARY KEY,
23     owner_id INTEGER NOT NULL REFERENCES AppUser(user_id) ON DELETE CASCADE,
24     duration_ms INTEGER NOT NULL CHECK (duration_ms BETWEEN 20000 AND 90000),
25     caption TEXT,
26     audio_track_id INTEGER REFERENCES AudioTrack(track_id) ON DELETE SET NULL,
27     uploaded_at TEXT NOT NULL
28 );
29
30 CREATE TABLE Hashtag (
31     hashtag_id INTEGER PRIMARY KEY,
32     text_normalised TEXT NOT NULL UNIQUE
33 );
34
35 CREATE TABLE VideoHashtag (
36     video_id INTEGER NOT NULL REFERENCES Video(video_id) ON DELETE CASCADE,
37     hashtag_id INTEGER NOT NULL REFERENCES Hashtag(hashtag_id) ON DELETE CASCADE,
38     PRIMARY KEY (video_id, hashtag_id)
39 );

```

Listing 4: Moderation lookup table and event log

```

1 CREATE TABLE ModerationState (
2     state_code TEXT PRIMARY KEY CHECK (state_code IN
3         ('pending','live','age_restricted','demoted','taken_down'))
4 );
5 INSERT INTO ModerationState VALUES
6     ('pending'),('live'),('age_restricted'),('demoted'),('taken_down');
7
8 CREATE TABLE VideoModerationEvent (
9     event_id INTEGER PRIMARY KEY,
10    video_id INTEGER NOT NULL REFERENCES Video(video_id) ON DELETE CASCADE,
11    state TEXT NOT NULL REFERENCES ModerationState(state_code)
12        ON DELETE RESTRICT,
13    decided_by_type TEXT NOT NULL CHECK (decided_by_type IN
14        ('classifier','reviewer')),
15    decided_by_id INTEGER,
16    decided_at TEXT NOT NULL
17 );

```

Listing 5: Social graph: follow, block, mute

```

1 CREATE TABLE Follow (
2     follower_id INTEGER NOT NULL REFERENCES AppUser(user_id) ON DELETE CASCADE,
3     followee_id INTEGER NOT NULL REFERENCES AppUser(user_id) ON DELETE CASCADE,
4     started_at TEXT NOT NULL,
5     ended_at TEXT,
6     PRIMARY KEY (follower_id, followee_id, started_at),
7     CHECK (follower_id != followee_id)
8 );
9
10 CREATE TABLE Block (

```

```

11     blocker_id INTEGER NOT NULL REFERENCES AppUser(user_id) ON DELETE CASCADE,
12     blocked_id INTEGER NOT NULL REFERENCES AppUser(user_id) ON DELETE CASCADE,
13     blocked_at TEXT NOT NULL,
14     PRIMARY KEY (blocker_id, blocked_id),
15     CHECK (blocker_id != blocked_id)
16 );
17
18 CREATE TABLE Mute (
19     muter_id INTEGER NOT NULL REFERENCES AppUser(user_id) ON DELETE CASCADE,
20     muted_id INTEGER NOT NULL REFERENCES AppUser(user_id) ON DELETE CASCADE,
21     muted_at TEXT NOT NULL,
22     PRIMARY KEY (muter_id, muted_id),
23     CHECK (muter_id != muted_id)
24 );

```

Listing 6: Watch telemetry: impressions, view segments, likes, retractions, shares, comments, signals

```

1 CREATE TABLE Impression (
2     impression_id INTEGER PRIMARY KEY,
3     user_id INTEGER NOT NULL REFERENCES AppUser(user_id) ON DELETE CASCADE,
4     video_id INTEGER NOT NULL REFERENCES Video(video_id) ON DELETE CASCADE,
5     ts TEXT NOT NULL,
6     feed_position INTEGER NOT NULL CHECK (feed_position >= 0),
7     model_version TEXT NOT NULL
8 );
9
10 CREATE TABLE ViewSegment (
11     segment_id INTEGER PRIMARY KEY,
12     impression_id INTEGER NOT NULL REFERENCES Impression(impression_id)
13         ON DELETE CASCADE,
14     started_at TEXT NOT NULL,
15     ended_at TEXT NOT NULL,
16     CHECK (ended_at >= started_at)
17 );
18
19 CREATE TABLE Like_ (
20     like_id INTEGER PRIMARY KEY,
21     user_id INTEGER NOT NULL REFERENCES AppUser(user_id) ON DELETE CASCADE,
22     video_id INTEGER NOT NULL REFERENCES Video(video_id) ON DELETE CASCADE,
23     liked_at TEXT NOT NULL,
24     retracted_at TEXT
25 );
26
27 CREATE TABLE Retraction (
28     retraction_id INTEGER PRIMARY KEY,
29     like_id INTEGER NOT NULL REFERENCES Like_(like_id) ON DELETE CASCADE,
30     retracted_at TEXT NOT NULL
31 );
32
33 CREATE TABLE Share (
34     share_id INTEGER PRIMARY KEY,
35     user_id INTEGER NOT NULL REFERENCES AppUser(user_id) ON DELETE CASCADE,
36     video_id INTEGER NOT NULL REFERENCES Video(video_id) ON DELETE CASCADE,
37     destination TEXT NOT NULL CHECK (destination IN
38         ('whatsapp', 'instagram', 'copied_link')),
39     shared_at TEXT NOT NULL
40 );
41
42 CREATE TABLE Comment (
43     comment_id INTEGER PRIMARY KEY,
44     user_id INTEGER NOT NULL REFERENCES AppUser(user_id) ON DELETE CASCADE,
45     video_id INTEGER NOT NULL REFERENCES Video(video_id) ON DELETE CASCADE,
46     body TEXT NOT NULL,
47     commented_at TEXT NOT NULL
48 );
49
50 CREATE TABLE SignalEvent (
51     signal_id INTEGER PRIMARY KEY,
52     user_id INTEGER NOT NULL REFERENCES AppUser(user_id) ON DELETE CASCADE,
53     video_id INTEGER NOT NULL REFERENCES Video(video_id) ON DELETE CASCADE,
54     signal_type TEXT NOT NULL CHECK (signal_type IN

```

```

55         ('not_interested','report','follow_from_feed')),
56         occurred_at TEXT NOT NULL
57     );

```

Listing 7: Agent layer: sessions, turns, tool calls, templates, pricing, usage, ratings, recommendations

```

1  CREATE TABLE PromptTemplateVersion (
2      template_id INTEGER NOT NULL,
3      version_no INTEGER NOT NULL,
4      text_body TEXT NOT NULL,
5      created_at TEXT NOT NULL,
6      PRIMARY KEY (template_id, version_no)
7  );
8
9  CREATE TABLE ModelPricePeriod (
10     model_id INTEGER NOT NULL,
11     priced_at TEXT NOT NULL,
12     valid_to TEXT,
13     input_rate REAL NOT NULL,
14     output_rate REAL NOT NULL,
15     cached_input_rate REAL,
16     PRIMARY KEY (model_id, priced_at)
17 );
18
19 CREATE TABLE AgentSession (
20     session_id INTEGER PRIMARY KEY,
21     user_id INTEGER NOT NULL REFERENCES AppUser(user_id) ON DELETE CASCADE,
22     started_at TEXT NOT NULL
23 );
24
25 CREATE TABLE Turn (
26     turn_id INTEGER PRIMARY KEY,
27     session_id INTEGER NOT NULL REFERENCES AgentSession(session_id)
28         ON DELETE CASCADE,
29     turn_index INTEGER NOT NULL,
30     user_message TEXT NOT NULL,
31     assistant_message TEXT NOT NULL,
32     template_id INTEGER NOT NULL,
33     template_version INTEGER NOT NULL,
34     model_id INTEGER NOT NULL,
35     priced_at TEXT NOT NULL,
36     temperature REAL NOT NULL,
37     occurred_at TEXT NOT NULL,
38     UNIQUE (session_id, turn_index),
39     FOREIGN KEY (template_id, template_version)
40         REFERENCES PromptTemplateVersion(template_id, version_no),
41     FOREIGN KEY (model_id, priced_at)
42         REFERENCES ModelPricePeriod(model_id, priced_at)
43 );
44
45 CREATE TABLE TurnUsage (
46     turn_id INTEGER PRIMARY KEY REFERENCES Turn(turn_id) ON DELETE CASCADE,
47     input_tokens INTEGER NOT NULL CHECK (input_tokens >= 0),
48     output_tokens INTEGER NOT NULL CHECK (output_tokens >= 0),
49     cached_tokens INTEGER NOT NULL CHECK (cached_tokens >= 0)
50 );
51
52 CREATE TABLE ToolCall (
53     call_id INTEGER PRIMARY KEY,
54     turn_id INTEGER NOT NULL REFERENCES Turn(turn_id) ON DELETE CASCADE,
55     parent_call_id INTEGER REFERENCES ToolCall(call_id) ON DELETE CASCADE,
56     tool_name TEXT NOT NULL,
57     arguments_json TEXT NOT NULL CHECK (json_valid(arguments_json)),
58     result_json TEXT CHECK (result_json IS NULL OR json_valid(result_json)),
59     latency_ms INTEGER NOT NULL CHECK (latency_ms >= 0),
60     errored INTEGER NOT NULL CHECK (errored IN (0,1)),
61     called_at TEXT NOT NULL
62 );
63
64 CREATE TABLE JudgeScore (
65     score_id INTEGER PRIMARY KEY,

```

```

66     turn_id INTEGER NOT NULL REFERENCES Turn(turn_id) ON DELETE CASCADE,
67     helpfulness REAL NOT NULL CHECK (helpfulness BETWEEN 0 AND 5),
68     groundedness REAL NOT NULL CHECK (groundedness BETWEEN 0 AND 5),
69     safety REAL NOT NULL CHECK (safety BETWEEN 0 AND 5),
70     scored_at TEXT NOT NULL
71 );
72
73 CREATE TABLE UserRating (
74     rating_id INTEGER PRIMARY KEY,
75     turn_id INTEGER NOT NULL REFERENCES Turn(turn_id) ON DELETE CASCADE,
76     thumbs TEXT NOT NULL CHECK (thumbs IN ('up','down')),
77     rated_at TEXT NOT NULL
78 );
79
80 CREATE TABLE Recommendation (
81     turn_id INTEGER NOT NULL REFERENCES Turn(turn_id) ON DELETE CASCADE,
82     position INTEGER NOT NULL CHECK (position >= 0),
83     video_id INTEGER NOT NULL REFERENCES Video(video_id) ON DELETE CASCADE,
84     PRIMARY KEY (turn_id, position)
85 );

```

3.2 C.2 Mapping Decisions

Video to Hashtag was a many to many relationship, so it became its own junction table, VideoHashtag, rather than a comma separated tag list stuffed into a text column on Video. A comma separated list would make it effectively impossible to look up every video carrying a given tag without scanning every single caption in the table.

ToolCall nesting became a plain self referencing parent_call_id column on the same table. A separate table per nesting level was considered, but the brief describes nesting that can go arbitrarily deep, and a fixed set of depth specific tables simply cannot cover an unbounded depth.

CreatorTierPeriod got its own table with valid_from and valid_to, rather than a single tier column sitting on Creator that gets overwritten every time a creator moves tiers. An overwritten column cannot answer what tier a creator held last March, which is exactly the kind of question Finance needs answered.

InferredInterest and InterestSuppression were split into two separate tables with different lifetimes, instead of one combined table with a suppressed flag column. The score is disposable and gets refreshed weekly, while suppression has to survive that refresh, and keeping both in one place risks the weekly refresh job accidentally wiping out a suppression setting.

Creator became its own table, one to one and optional against AppUser, rather than adding tier and creator flag columns directly onto AppUser. This keeps creator specific temporal history out of the main user table, which the majority of users never even touch since most accounts are never creators.

Like_ and Retraction ended up as a nullable retracted_at column on Like_ plus a fully separate Retraction row, instead of a single is_retracted boolean flag. The brief specifically wants retraction treated as its own independent signal with its own timestamp, and a boolean column cannot represent that as a distinct event with a distinct time.

TurnUsage was added as its own table, separate from Turn, specifically to hold token counts. This was not part of the earliest design and only got added once Deliverable F needed real, queryable cost numbers per turn, at which point it became clear the schema had no way at all to answer a cost question without it.

3.3 C.3 Relational Algebra

F2, watch hours and completion rate per creator, restricted to live clips only, where every creator must appear even with zero activity.

$$\Pi_{creator_id, sum_watch, avg_completion} \left(\gamma_{creator_id; SUM(watch_ms), AVG(completed)} (Creator \bowtie \sigma_{state='live'}(Video) \bowtie ViewSegment) \right)$$

Here $Creator \bowtie \sigma_{state='live'}(Video)$ is a left outer join, written as $Creator \overset{O}{\bowtie} \sigma_{state='live'}(Video)$, so that a creator with no live videos at all still appears in the result with zero values, rather than disappearing the way a plain inner join would drop that creator entirely.

F3, videos with no audio track attached.

$$\sigma_{audio_track_id \text{ IS NULL}}(Video)$$

F12, sessions where the agent recommended a clip the user actually watched to completion, keeping the shelf position.

$$\Pi_{session_id, turn_id, position, video_id} \left(Recommendation \bowtie \sigma_{completed}(ViewSegment \bowtie Impression) \bowtie Turn \right)$$

Rewrite of F2, pushing a selection below the join.

Before the rewrite, the selection restricting to live videos happens on Video alone, and then Video is joined against the entire ViewSegment table before the sum and grouping run.

After the rewrite, a selection restricting ViewSegment to the relevant reporting window is pushed down so it runs before the join takes place, meaning both sides of the join are already smaller by the time the join itself executes.

Why this should be cheaper: filtering ViewSegment down before the join means the join only ever has to combine rows that are already restricted on both sides, instead of joining the full unrestricted table first and only cutting the result down afterward. This is less total work for the engine to do before grouping and aggregation even begin.

4 Deliverable D - Normalisation

Four relations were picked for the full writeup. Turn covers the agent layer, CreatorTierPeriod and VideoModerationEvent are both temporal in nature, and TurnUsage is the one that actually needed decomposing, since it is the clearest place in the whole schema where a dependency does not sit on a key.

Turn

Turn holds turn_id, session_id, turn_index, template_id, template_version, model_id, temperature, occurred_at.

First dependency: turn_id determines every other attribute in the row. This makes sense because a turn is created exactly once, answered by exactly one model using exactly one template, at one specific instant, and none of that changes afterward.

Second dependency: session_id together with turn_index also determines turn_id. Turns inside a session are numbered in strict order, so knowing the session and the position tells you exactly which turn it is, since there is only ever one turn at position 3 of session 4471.

Working through the closures, the closure of turn_id alone reaches every attribute through the first dependency. The closure of session_id plus turn_index reaches turn_id through the second dependency, and once turn_id is known, the first dependency fires and gives everything else too, so this pair also reaches the whole relation.

That gives two candidate keys, turn_id by itself and session_id plus turn_index together. Both dependencies have a left side that is a full candidate key, so this relation sits comfortably in BCNF with nothing left to decompose.

CreatorTierPeriod

Holds creator_id, valid_from, valid_to, tier.

The only dependency here is that creator_id together with valid_from determines valid_to and tier. A creator can only be on one tier at a time, so knowing the creator and when a period started pins down exactly when it ended and what tier applied during it.

The closure of creator_id plus valid_from reaches the whole relation, so that pair is the only candidate key, and since the one dependency already has that pair as its left side, this relation is already in BCNF.

It is worth being honest that no functional dependency analysis catches the separate rule that two tier periods for the same creator should never overlap in time. That is a genuine constraint, but it is not something a functional dependency can express on its own. It needs enforcement somewhere else entirely, either in application code or through a database trigger, which is exactly what shows up later as one of the bonus items.

VideoModerationEvent

Holds event_id, video_id, state, decided_by_type, decided_by_id, decided_at.

The only dependency is that event_id determines everything else, since each moderation decision is its own independent event, and once you know which specific event it is, everything about it is already fixed.

It is deliberately not claimed that video_id determines state, since one video accumulates many events over its life with different states at different times, so that would not even be a valid dependency to write down in the first place.

Event_id reaches the whole relation on its own, so it is the only candidate key, and since it is the left side of the only dependency, this relation is BCNF as well.

TurnUsage

Holds turn_id, model_id, priced_at, input_tokens, output_tokens, cached_tokens, input_rate, output_rate.

First dependency: turn_id determines model_id, priced_at, and the three token counts. A turn is billed exactly once, by exactly one model, at one instant, and the token counts never change after the fact.

Second dependency: `model_id` together with `priced_at` determines `input_rate` and `output_rate`. A vendor's price for a given model holds steady for whatever interval it is valid over, so any two turns hitting the same model during the same price period always pay the same rate, regardless of which specific turn is asking.

The closure of `turn_id` reaches everything: the first dependency supplies `model_id`, `priced_at`, and the token counts, and once `model_id` and `priced_at` are both known, the second dependency fires and adds the two rates as well. So `turn_id` is a candidate key.

The closure of `model_id` plus `priced_at` only reaches `input_rate` and `output_rate` through the second dependency, and nothing else adds anything further from there, so this closure stops well short of the full relation. That means `model_id` plus `priced_at` is not a superkey.

This is a genuine BCNF violation, not just an awkward edge case, since price depends purely on the model and the pricing period, never on which particular turn happens to be asking.

The relation was split into two: R1 keeps `turn_id`, `model_id`, `priced_at`, and the three token counts, while R2 keeps `model_id`, `priced_at`, `input_rate`, and `output_rate`.

Checking losslessness, the two share `model_id` and `priced_at`, and that exact pair is the primary key of R2, so joining R1 back to R2 reconstructs the original table precisely, with nothing extra and nothing missing.

Both of the original dependencies survive the split cleanly too. The first sits entirely inside R1, and the second sits entirely inside R2, so nothing was lost in the process of decomposing.

The Denormalisation That Was Not Done

There was a temptation to add a `current_state` column directly onto `Video`, so that Trust and Safety could read a video's moderation status in a single lookup instead of always pulling the latest row out of `VideoModerationEvent`.

That would speed up exactly the query the brief names in section 2.6, current moderation state right now in one lookup, which against a pure event log needs an order by plus a limit, or a window function, just to find the latest row.

The anomaly this would open up is the exact same one the brief already describes happening inside the founders' own system. A moderation decision would then need two separate writes at once, a new history row and an update to the cached column, and if one write lands without the other, say a process is killed mid request, the two values disagree, with history saying taken down while the cached column still says live, and nothing in the schema would ever catch that.

Doing it honestly would require either a trigger that updates `Video` every time a new event lands, or wrapping both writes inside one transaction in application code. Even with a trigger, SQLite only fires triggers per row, so the trigger logic would always need to specifically grab the newest event, since a late arriving row landing out of order could still leave the cached value wrong.

Given the real risk of drift, and given that the brief calls this exact failure out by name, the column was left out entirely. Current state stays something that is always derived fresh from the event log, which is also exactly what the `v_video.current_state` view in Deliverable G does.

Checking This Against a Real Database

Rather than leaving all of the above asserted purely on paper, a small SQLite database covering these four relations was actually built, using `schema_d.demo.sql` and `verify_normalisation.py`, and real inserts were run against it.

Listing 8: `schema_d.demo.sql`, the working demo schema for Deliverable D

```
1 PRAGMA foreign_keys = ON;
2
3 DROP TABLE IF EXISTS Turn;
4 DROP TABLE IF EXISTS CreatorTierPeriod;
5 DROP TABLE IF EXISTS VideoModerationEvent;
6 DROP TABLE IF EXISTS TurnUsage;
7 DROP TABLE IF EXISTS ModelPricePeriod;
8
9 CREATE TABLE Turn (
10     turn_id      INTEGER PRIMARY KEY,
```

```

11     session_id    INTEGER NOT NULL,
12     turn_index    INTEGER NOT NULL,
13     template_id   INTEGER,
14     template_version INTEGER,
15     model_id      INTEGER,
16     temperature   REAL,
17     occurred_at   TEXT NOT NULL,
18     UNIQUE (session_id, turn_index)
19 );
20
21 CREATE TABLE CreatorTierPeriod (
22     creator_id    INTEGER NOT NULL,
23     valid_from    TEXT NOT NULL,
24     valid_to      TEXT,
25     tier           TEXT NOT NULL,
26     PRIMARY KEY (creator_id, valid_from)
27 );
28
29 CREATE TABLE VideoModerationEvent (
30     event_id      INTEGER PRIMARY KEY,
31     video_id      INTEGER NOT NULL,
32     state         TEXT NOT NULL,
33     decided_by_type TEXT NOT NULL,
34     decided_by_id INTEGER,
35     decided_at    TEXT NOT NULL
36 );
37
38 CREATE TABLE TurnUsage (
39     turn_id       INTEGER PRIMARY KEY,
40     model_id      INTEGER NOT NULL,
41     priced_at     TEXT NOT NULL,
42     input_tokens  INTEGER NOT NULL,
43     output_tokens INTEGER NOT NULL,
44     cached_tokens INTEGER NOT NULL
45 );
46
47 CREATE TABLE ModelPricePeriod (
48     model_id      INTEGER NOT NULL,
49     priced_at     TEXT NOT NULL,
50     input_rate    REAL NOT NULL,
51     output_rate   REAL NOT NULL,
52     PRIMARY KEY (model_id, priced_at)
53 );

```

Inserting a second Turn row with a duplicate turn_id got rejected with a unique constraint failure, confirming that key. A second row with a duplicate session_id and turn_index pair but a different turn_id was also rejected, confirming the second key found through the closure computation. The same pattern held for a duplicate creator_id plus valid_from on CreatorTierPeriod, and for a duplicate model_id plus priced_at on ModelPricePeriod.

For the TurnUsage split specifically, one row's turn level facts were inserted into R1 and its price period facts into R2, then joined back together on model_id and priced_at and compared against the original row. The reconstructed row matched exactly, same token counts and same rates, confirming the decomposition is lossless in actual practice, not merely by the intersection-is-a-key argument on paper.

Listing 9: The key section of verify_normalisation.py, checking the lossless join for real

```

1 cur.execute("""
2     SELECT tu.turn_id, tu.model_id, tu.priced_at, tu.input_tokens,
3           tu.output_tokens, tu.cached_tokens, mp.input_rate, mp.output_rate
4     FROM TurnUsage tu
5     JOIN ModelPricePeriod mp ON tu.model_id = mp.model_id
6                             AND tu.priced_at = mp.priced_at
7     WHERE tu.turn_id = 1
8 """)
9 row = cur.fetchone()
10 expected = (1, 5, '2026-03-01', 1200, 340, 50, 0.0005, 0.0015)
11 assert row == expected, f"lossless join check failed: got {row}"
12 print("Test 5 passed: reconstructed row matches expected exactly")

```

5 Deliverable E - Physical Implementation

Type Affinity Check

A small test table was created with a column declared INTEGER, and the text value hello was then inserted into it. SQLite did not reject this insert. Running typeof on the stored value showed it had been kept as text, not converted into a number.

Listing 10: Proving SQLite type affinity is a hint, not an enforcement

```
1 INSERT INTO TypeAffinityTest (id, strict_int) VALUES (1, 'hello');
2 SELECT id, strict_int, typeof(strict_int) FROM TypeAffinityTest;
3 -- result: 1 | hello | text
```

This means the type name written in the schema is mostly a hint, not a hard rule. Most of the actual correctness in this schema comes from CHECK constraints, not from the column type names themselves. STRICT tables were not used here, since several columns use TEXT for timestamps that work fine under normal SQLite typing, and switching to STRICT would need extra rewriting across the schema for no real practical gain.

Primary Keys, Foreign Keys, ON DELETE

Every table has a primary key, either a surrogate integer id or a natural composite key wherever the brief made that clear, for example Turn using session_id plus turn_index as a second key alongside turn_id.

ON DELETE was chosen per relationship rather than applying the same rule everywhere. CASCADE is used where the child row has no meaning without its parent, for example Handle, Like_, Comment, and ToolCall all cascade when the parent user, video, or turn is deleted. SET NULL is used on Video.audio_track_id, since a video losing its audio track should still exist, just without a track attached. RESTRICT is used on InferredInterest.category_id, since deleting a category while users still hold that inferred interest should not be allowed to happen silently.

Closed Value Sets

Moderation state is a lookup table, ModerationState, with a foreign key from VideoModerationEvent, rather than a plain CHECK list. This is because the brief notes that Trust and Safety may want to add new states later, and a lookup table lets that happen without touching the schema at all.

Account state, share destination, and creator tier are instead plain CHECK constraints on TEXT columns. These are small, fixed sets that are not expected to grow, so a full lookup table would just add extra joins for no real benefit.

Handle Uniqueness

Listing 11: The partial unique index enforcing A9 declaratively

```
1 CREATE UNIQUE INDEX idx_handle_active_unique
2   ON Handle(handle COLLATE NOCASE)
3   WHERE ended_at IS NULL;
```

This single index covers both halves of the rule at once. Uniqueness is case insensitive through NOCASE, and it only applies to currently active handle rows because of the WHERE clause, not to old, already ended ones. NOCASE only folds ASCII letters, so a handle written in a non Latin script has no upper or lower case to fold in the first place, meaning NOCASE does not break it, but it also would not catch two different Unicode encodings of what visually looks like the same text.

Timestamps

ISO 8601 text was used everywhere, for example 2026-03-03T14:32:07Z, which sorts correctly as plain text for range queries. For the rolling window queries in F9 and F10, SQLite's window frame needs a plain number rather than a date, so the timestamp has to be converted through julianday first before it can be used inside a RANGE BETWEEN clause.

NOT NULL Audit, A Few Examples

Video.audio_track_id being null means the video genuinely has no music, not that the track is simply unknown. AppUser.deletion_requested_at being null means deletion was never requested, not that the user declined it. ToolCall.parent_call_id being null means this call is not nested under another call, not that the nesting information was never recorded.

5.1 E.2 Constraint Trace Table

Assum.	Rule	Enforced by	Where
A1	a user may hold both a phone and a Google account at once	CHECK requiring at least one identity to be non null, without forbidding both	AppUser
A2	previous handles are retained as history	Handle is its own table keyed on user_id plus started_at, not a single overwritten column	Handle
A3	suppression must survive the weekly refresh	InterestSuppression lives in a table separate from InferredInterest, so the refresh job never touches it	InferredInterest, InterestSuppression
A4	a follow that ends and restarts is two periods, not one	Follow's primary key includes started_at, so a new follow after an old one ended always inserts a new row	Follow
A5	the same signal type can be logged more than once per user and video	no uniqueness constraint on user_id, video_id, signal_type, only signal_id as primary key	SignalEvent
A6	a share has exactly one destination	destination is a plain single valued CHECK column, not a separate multi valued table	Share
A7	a view is derived, not stored directly	no View table exists at all, F queries derive it from ViewSegment against Video.duration_ms	ViewSegment, Video
A8	a video's owner never changes after creation	no update statement anywhere in the app touches Video.owner_id	Video
A9	handle unique, only among active accounts	partial unique index with COLLATE NOCASE and a WHERE ended_at IS NULL clause	Handle
A10	a nested tool call has exactly one parent	parent_call_id is a plain nullable self referencing foreign key, not a join table	ToolCall
A11	a turn is billed at the price active at that time	Turn.priced_at foreign keys directly into ModelPricePeriod's own composite key	Turn, ModelPricePeriod
A12	current moderation state is always derived, never separately stored	no current_state column exists on Video at all, v_video_current_state derives it from the log	VideoModerationEvent

Most of these rules are enforced declaratively through constraints, indexes, or foreign keys, which SQLite can check on every write with no help from application code. A handful, particularly A4 and A8, rely partly on the application never writing an update that would violate them, since SQLite has no declarative way to forbid an UPDATE statement outright. That is a real, honestly stated limitation rather than something to gloss over.

5.2 E.3 Data Generator

generate_data.py builds the core data: users, creators, videos, moderation events, impressions, view segments, likes, retractions, shares, comments, signals, follows, agent sessions, turns, and tool calls, all inside one transaction per stage so the load runs quickly even at scale.

A series of patch scripts run on top of that base generator, each one fixing a specific problem that was found while actually testing the Deliverable F queries against the generated data, rather than trusting that a script running without a Python exception meant the data was correct. These add hashtags parsed from video captions, inferred interests together with suppression rows, blocks and mutes, and recommendations linking agent turns to real videos and outcomes. They also fix zero duration view segments, fix instantly retracted likes, add real multi step moderation histories instead of a single decision per video, fix a uniform activity bug that made almost every user look active every single day, backfill token usage numbers, add real nested tool calls, and fix recommendation linked watches that had been capped at a fixed short duration regardless of the actual clip length.

Row counts after all patches, using seed 2100000 at the default generation scale, come out to roughly 5000 users, 20000 videos, over 300000 impressions, around 107000 view segments, about 8300 likes, 815 retractions, 7004 turns, over 7000 tool calls, about 8600 recommendations, 20000 rows in the video to hashtag junction table, and around 12500 inferred interest rows.

The resulting funnel shape matches what the brief describes qualitatively. Roughly 35 percent of impressions turn into an actual view segment, and only around 8 percent of view segments go on to produce a like. Most turns never receive a judge score or a user rating at all, which also matches the rarity the brief describes in section 2.5.

5.3 E.4 JSON Use

ToolCall.arguments_json is the only JSON column anywhere in the schema, guarded by a CHECK that the stored text is syntactically valid JSON.

Listing 12: The JSON guard on ToolCall

```
1 arguments_json TEXT NOT NULL CHECK (json_valid(arguments_json)),  
2 result_json TEXT CHECK (result_json IS NULL OR json_valid(result_json)),
```

This choice is defensible because different tools, search videos, get user history, and fetch trending audio, each take a differently shaped set of arguments. Forcing this into ordinary columns would mean either a single table with one column per possible argument across every tool, mostly sitting empty for any given row, or a completely separate table per tool, which does not scale cleanly as new tools get added over time.

What is genuinely lost by taking this route is real, and worth stating plainly. No foreign key can reach into a field sitting inside the JSON text, so a video_id mentioned inside the arguments cannot be checked against the real Video table by the database itself. No CHECK constraint can validate the interior shape either, beyond confirming that the text is syntactically valid JSON at all, not that it actually carries the right fields for that specific tool.

6 Deliverable F - Queries

Thirteen queries were built, one at a time, each run against the live populated database and checked against a plausible expected shape, not just checked for the absence of a SQL error. Several of them exposed real problems in the generated data along the way, and those are documented honestly below rather than hidden.

F1 - Most used audio tracks in the last 7 days

```
1 SELECT
2     at.track_id,
3     at.source,
4     COUNT(DISTINCT v.video_id) AS distinct_video_count
5 FROM AudioTrack at
6 JOIN Video v ON v.audio_track_id = at.track_id
7 WHERE v.uploaded_at >= (
8     SELECT datetime(MAX(uploaded_at), '-7 days') FROM Video
9 )
10 GROUP BY at.track_id, at.source
11 ORDER BY distinct_video_count DESC
12 LIMIT 10;
```

The window is computed relative to the latest upload timestamp in the table rather than the real current date, since the generated data does not necessarily reach all the way to today. Returned 10 rows as expected from the LIMIT, each showing a track id, its source, and how many distinct videos used it in that trailing week. Runtime was consistently fast, in the 10 to 30 millisecond range, since the table is small relative to Video.

F2 - Watch hours and completion rate per creator, live videos only

```
1 SELECT
2     c.user_id AS creator_id,
3     COALESCE(SUM(
4         (julianday(vs.ended_at) - julianday(vs.started_at)) * 86400.0
5     ) / 3600.0, 0) AS total_watch_hours,
6     COALESCE(AVG(
7         CASE
8             WHEN (julianday(vs.ended_at) - julianday(vs.started_at)) * 86400000.0
9                 >= v.duration_ms
10            THEN 1.0 ELSE 0.0
11        END
12    ), 0) AS mean_completion_rate
13 FROM Creator c
14 LEFT JOIN Video v
15     ON v.owner_id = c.user_id
16 LEFT JOIN VideoModerationEvent vme
17     ON vme.video_id = v.video_id
18     AND vme.state = 'live'
19 LEFT JOIN Impression i
20     ON i.video_id = v.video_id
21 LEFT JOIN ViewSegment vs
22     ON vs.impression_id = i.impression_id
23 GROUP BY c.user_id;
```

This one had a real problem twice over. First, an earlier draft used a correlated subquery inside the ViewSegment join condition, checking `impression_id IN (SELECT ... WHERE video_id = v.video_id)`, and that version ran for minutes before it had to be interrupted. Rewriting it into two plain LEFT JOINs, Impression then ViewSegment, fixed the performance problem entirely. Second, once the query actually ran, every creator showed almost zero watch hours and a completion rate of exactly zero, because every ViewSegment in the original generated data had `started_at` equal to `ended_at`, meaning zero duration for every single segment. This was fixed with `fix_view_durations.py`, which recomputes a real `ended_at` using `strftime` with the same T-formatted pattern as the rest of the schema, specifically avoiding SQLite's `datetime` function, which reformats timestamps into a space separated form that breaks a plain text comparison against the T-formatted originals. After the fix, all 750 creators returned a row, with plausible positive watch hours and completion rates. Runtime was around 2.5 to 2.8 seconds, the slowest query alongside F12, since it joins across four tables at full scale.

F3 - Videos with no audio track

```
1  -- version using NOT IN
2  SELECT video_id
3  FROM Video
4  WHERE audio_track_id NOT IN (
5      SELECT track_id FROM AudioTrack
6  );
7
8  -- version using NOT EXISTS
9  SELECT v.video_id
10 FROM Video v
11 WHERE NOT EXISTS (
12     SELECT 1 FROM AudioTrack at WHERE at.track_id = v.audio_track_id
13 );
```

These two look equivalent but are not, because of how SQL NULL comparisons work. The NOT IN version returned 0 rows, while the NOT EXISTS version correctly returned 5984 rows. The reason is that audio_track_id is nullable, and once the subquery for NOT IN evaluates a comparison against a NULL, the whole NOT IN condition becomes UNKNOWN rather than TRUE for that row, silently excluding every video with a NULL audio_track_id from ever matching, which is exactly the set of rows this query is supposed to find. NOT EXISTS does not have this problem because it checks for the existence of a matching row rather than comparing values directly, so a NULL audio_track_id on the outer row is handled correctly and the row is still returned.

F4 - Likes retracted within 60 seconds

```
1  SELECT
2      l.user_id,
3      l.video_id,
4      l.liked_at,
5      l.retracted_at,
6      (julianday(l.retracted_at) - julianday(l.liked_at)) * 86400.0
7      AS seconds_between
8  FROM Like_1
9  WHERE l.retracted_at IS NOT NULL
10 AND (julianday(l.retracted_at) - julianday(l.liked_at)) * 86400.0 <= 60;
```

The first run of this query returned 815 out of 815 retractions, meaning every single retraction in the data happened within the 60 second window, which was technically correct but pedagogically meaningless, since it turned out every retraction had retracted_at set equal to liked_at in the base generator. This was patched with fix_retraction_gaps.py, giving 60 percent of retractions a 1 to 55 second gap and the remaining 40 percent a 120 to 7200 second gap, using the same strftime pattern as the ViewSegment fix to avoid the same timestamp formatting bug. After the fix, the query returned 487 rows out of the full 815 retractions, a genuinely meaningful filtered subset rather than the entire table.

F5 - Videos tagged with a specific hashtag

```
1  SELECT video_id, caption
2  FROM Video
3  WHERE instr(
4      ' ' || lower(replace(replace(caption, '.', ' '), ',', ' ')) || ' ',
5      ' #' || lower(:target_tag) || ' '
6  ) > 0
7  OR lower(caption) LIKE '%#' || lower(:target_tag);
```

This searches captions directly with string functions rather than joining through the VideoHashtag junction table, since the assignment brief for this specific query is framed around free text caption matching. Bound to the parameter tag5, it returned 400 matching videos.

F6 - Shown but never engaged, and duplicate handling with UNION

```
1  WITH shown AS (
2      SELECT DISTINCT i.user_id
```

```

3      FROM Impression i
4      JOIN Video v ON v.video_id = i.video_id
5      WHERE v.owner_id = :creator_id
6  ),
7  engaged AS (
8      SELECT l.user_id FROM Like_ l JOIN Video v ON v.video_id = l.video_id
9          WHERE v.owner_id = :creator_id
10     UNION
11     SELECT c.user_id FROM Comment c JOIN Video v ON v.video_id = c.video_id
12         WHERE v.owner_id = :creator_id
13     UNION
14     SELECT s.user_id FROM Share s JOIN Video v ON v.video_id = s.video_id
15         WHERE v.owner_id = :creator_id
16     UNION
17     SELECT se.user_id FROM SignalEvent se JOIN Video v ON v.video_id = se.video_id
18         WHERE v.owner_id = :creator_id
19 )
20 SELECT user_id FROM shown
21 EXCEPT
22 SELECT user_id FROM engaged;
23
24 -- comparing UNION and UNION ALL directly
25 SELECT user_id FROM Like_
26 UNION
27 SELECT user_id FROM Comment
28 UNION
29 SELECT user_id FROM Share;
30 -- 4503 rows
31
32 SELECT user_id FROM Like_
33 UNION ALL
34 SELECT user_id FROM Comment
35 UNION ALL
36 SELECT user_id FROM Share;
37 -- 12089 rows

```

The main EXCEPT query returned 342 users who were shown a given creator's videos but never engaged with any of them in any of the four tracked ways. The separate UNION versus UNION ALL comparison makes the difference concrete: plain UNION deduplicates across all three source tables and returned 4503 distinct user ids, while UNION ALL keeps every row from every table without removing duplicates and returned 12089 rows, since the same user id can legitimately appear in more than one of Like_, Comment, and Share.

F7 - Sessions costing more than a threshold, grouped by template

```

1  SELECT
2      t.session_id,
3      t.template_id,
4      t.template_version,
5      SUM(tu.input_tokens * mp.input_rate
6          + tu.output_tokens * mp.output_rate
7          + tu.cached_tokens * mp.cached_input_rate) AS session_template_cost
8  FROM Turn t
9  JOIN TurnUsage tu ON tu.turn_id = t.turn_id
10 JOIN ModelPricePeriod mp
11     ON mp.model_id = t.model_id AND mp.priced_at = t.priced_at
12 WHERE t.occurred_at >= '2026-08-01T00:00:00Z'
13     AND t.occurred_at < '2026-09-01T00:00:00Z'
14 GROUP BY t.session_id, t.template_id, t.template_version
15 HAVING session_template_cost > 2.0
16 ORDER BY session_template_cost DESC;

```

This is the query that motivated adding TurnUsage to the schema in the first place, since without token counts there was no way to compute a real per turn cost at all. At the 2.0 threshold it returned 265 rows, with costs across the returned set ranging from about 0.07 to 4.11 and averaging around 0.94.

F8 - Videos with more than two moderation transitions

```

1 SELECT
2     video_id,
3     COUNT(*) AS transition_count,
4     group_concat(state, ' -> ' ORDER BY decided_at) AS transition_sequence
5 FROM VideoModerationEvent
6 GROUP BY video_id
7 HAVING COUNT(*) > 2
8 ORDER BY transition_count DESC;

```

Getting believable data for this one took two failed attempts. The first attempt picked a random day offset and a fully independent random hour for each step in a video's moderation history, which produced non monotonic and sometimes nonsensical sequences, for example pending then live then taken_down then demoted, or a sequence that started on an invalid first state entirely. The second attempt tried manual hour and day rollover arithmetic and still occasionally produced incorrect ordering. The fix that finally worked, in `patch_moderation_history.py`, used Python's real datetime and timedelta objects to guarantee strictly increasing timestamps across a chosen 3 to 4 step sequence drawn from five plausible moderation state sequences. After the fix, the query returned 1600 rows, and every single one carried exactly 4 transitions, matching one of the five valid sequences used by the generator.

F9 - Longest daily activity streak per user

```

1 WITH active_days AS (
2     SELECT DISTINCT user_id, date(ts) AS active_date
3     FROM Impression
4 ),
5 numbered AS (
6     SELECT
7         user_id,
8         active_date,
9         ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY active_date) AS rn
10    FROM active_days
11 ),
12 grouped AS (
13     SELECT
14         user_id,
15         active_date,
16         date(active_date, '-' || rn || ' days') AS streak_group
17    FROM numbered
18 ),
19 streaks AS (
20     SELECT
21         user_id,
22         streak_group,
23         COUNT(*) AS streak_length
24    FROM grouped
25    GROUP BY user_id, streak_group
26 )
27 SELECT
28     user_id,
29     MAX(streak_length) AS longest_streak
30 FROM streaks
31 GROUP BY user_id
32 ORDER BY longest_streak DESC;

```

This uses the row number gap trick for detecting consecutive runs: subtracting the row number from the date collapses every consecutive run of dates onto the same streak_group value, since the gap between the date and the row number stays constant only while the dates are consecutive. A distribution check on the first run of this query showed 4991 out of 5000 users with a longest streak of exactly 7, which is implausible for a full user base. The cause was that the base generator picked a user uniformly at random for every single one of over 300000 impressions, and with roughly 60 impressions landing on each user on average spread across only 7 possible days, nearly everyone touched all 7 days purely through sheer volume. This was fixed with `fix_user_activity_pattern.py`, which assigns each user a personal weighted random count of active days, favoring low activity, and then reassigns each impression's date to match that personal pattern while preserving the original time of day. After the fix, the query returned all 5000 users with a much more plausible distribution, 2636 users maxing out at a streak of 1, 1258 at 2, 540 at 3, 262 at 4, 120 at 5, and smaller counts trailing off at 6 and 7.

F10 - Rolling 7 day watch time and week over week change

```
1  WITH daily_watch AS (  
2      SELECT  
3          v.owner_id AS creator_id,  
4          date(i.ts) AS watch_date,  
5          SUM((julianday(vs.ended_at) - julianday(vs.started_at)) * 86400.0)  
6              AS watch_seconds  
7      FROM ViewSegment vs  
8      JOIN Impression i ON i.impression_id = vs.impression_id  
9      JOIN Video v ON v.video_id = i.video_id  
10     GROUP BY v.owner_id, date(i.ts)  
11 ),  
12 rolling AS (  
13     SELECT  
14         creator_id,  
15         watch_date,  
16         watch_seconds,  
17         SUM(watch_seconds) OVER (  
18             PARTITION BY creator_id  
19             ORDER BY julianday(watch_date)  
20             RANGE BETWEEN 6 PRECEDING AND CURRENT ROW  
21         ) AS rolling_7day_watch_seconds  
22     FROM daily_watch  
23 )  
24 SELECT  
25     creator_id,  
26     watch_date,  
27     rolling_7day_watch_seconds,  
28     rolling_7day_watch_seconds - LAG(rolling_7day_watch_seconds, 7) OVER (  
29         PARTITION BY creator_id ORDER BY julianday(watch_date)  
30     ) AS week_over_week_change  
31 FROM rolling  
32 ORDER BY creator_id, watch_date;
```

SQLite has no native INTERVAL type, so the RANGE BETWEEN frame needs a plain numeric offset rather than a date literal, which is why watch_date is converted through julianday before being used as the ORDER BY expression inside the window. The query returned 10986 creator-day pairs in total, each carrying its own rolling 7 day watch time and, where 7 prior days of history existed, a week over week change computed with LAG.

F11 - Recursive tool call tree for one session

```
1  WITH RECURSIVE call_tree AS (  
2      SELECT  
3          call_id, turn_id, parent_call_id, tool_name, called_at, 0 AS depth  
4      FROM ToolCall  
5      WHERE turn_id IN (SELECT turn_id FROM Turn WHERE session_id = 389)  
6          AND parent_call_id IS NULL  
7  
8      UNION ALL  
9  
10     SELECT  
11         tc.call_id, tc.turn_id, tc.parent_call_id, tc.tool_name,  
12         tc.called_at, ct.depth + 1  
13     FROM ToolCall tc  
14     JOIN call_tree ct ON tc.parent_call_id = ct.call_id  
15 )  
16 SELECT call_id, turn_id, parent_call_id, tool_name, depth  
17 FROM call_tree  
18 ORDER BY turn_id, depth, call_id;
```

Session 389 was picked specifically because it turned out to be the most deeply nested session in the generated data. The anchor member of the recursive query picks up every root level call belonging to that session's turns, and the recursive member walks down one level of nesting at a time by joining each call back to its parent in the previous depth. This returned 19 rows total for session 389, with nesting depth going as deep as three levels below the root calls.

F12 - Recommendations that were watched to completion

```
1 SELECT
2     t.session_id,
3     r.turn_id,
4     r.position,
5     r.video_id,
6     MAX(
7         CASE
8             WHEN (julianday(vs.ended_at) - julianday(vs.started_at)) * 86400000.0
9                 >= v.duration_ms
10            THEN 1 ELSE 0
11        END
12    ) AS watched_to_completion
13 FROM Recommendation r
14 JOIN Turn t ON t.turn_id = r.turn_id
15 JOIN AgentSession s ON s.session_id = t.session_id
16 JOIN Video v ON v.video_id = r.video_id
17 JOIN Impression i
18     ON i.video_id = r.video_id
19     AND i.user_id = s.user_id
20 JOIN ViewSegment vs
21     ON vs.impression_id = i.impression_id
22 GROUP BY t.session_id, r.turn_id, r.position, r.video_id
23 HAVING watched_to_completion = 1;
```

This query returned 0 rows on the very first run, even though all 8589 recommendations existed and the joins themselves were confirmed to be structurally correct using two separate diagnostic check queries. The real cause was that every one of the 2535 ViewSegments linked to a recommendation had been hardcoded in the generator to a fixed 20 second watch duration, regardless of the actual video's duration_ms, which typically ran between 20 and 90 seconds, making completion structurally impossible for almost every one of them. This was fixed with fix_recommendation_watches.py, giving 35 percent of these specific segments a full or near full completion watch and the remaining 65 percent a right skewed partial watch tied to the video's real duration. After the fix, the query returned 802 rows.

F13 - High judge score but a thumbs down from the user

```
1 SELECT
2     t.turn_id,
3     t.session_id,
4     js.helpfulness,
5     js.groundedness,
6     js.safety,
7     ur.thumbs
8 FROM Turn t
9 JOIN JudgeScore js ON js.turn_id = t.turn_id
10 JOIN UserRating ur ON ur.turn_id = t.turn_id
11 WHERE js.helpfulness > 4
12        AND ur.thumbs = 'down';
```

This is deliberately looking for the disagreement case between an automated judge and a real user, where the model rated its own helpfulness highly but the actual user still gave a thumbs down. It returned exactly 3 rows, which is a small but genuine result given how rare it is for both a judge score and a user rating to exist on the same turn at all.

7 Deliverable G - Views and Transactions

7.1 Views

Five views were built, one per consumer named in section 2.6 of the brief, and every one of them was actually run against the populated database rather than only checked by reading the SQL.

Listing 13: v_public_profile, for the mobile client

```
1 DROP VIEW IF EXISTS v_public_profile;
2 CREATE VIEW v_public_profile AS
3 SELECT
4     u.user_id,
5     h.handle,
6     u.display_name,
7     (
8         SELECT COUNT(*)
9         FROM Follow f
10        WHERE f.followee_id = u.user_id
11              AND f.ended_at IS NULL
12     ) AS follower_count
13 FROM AppUser u
14 JOIN Handle h
15     ON h.user_id = u.user_id
16     AND h.ended_at IS NULL
17 WHERE u.account_state = 'active';
```

This gives handle, display name, and a follower count worked out by counting active follow rows. It keeps out phone and email entirely, and it drops anyone deactivated or inside their deletion window by filtering on account_state equal to active. Row count came out to 4595, matching the active user count exactly.

Listing 14: v_video_current_state, for Trust and Safety

```
1 DROP VIEW IF EXISTS v_video_current_state;
2 CREATE VIEW v_video_current_state AS
3 SELECT video_id, state, decided_by_type, decided_at
4 FROM (
5     SELECT
6         video_id, state, decided_by_type, decided_at,
7         ROW_NUMBER() OVER (
8             PARTITION BY video_id ORDER BY decided_at DESC
9         ) AS rn
10    FROM VideoModerationEvent
11 )
12 WHERE rn = 1;
```

This grabs the latest row per video out of VideoModerationEvent using row_number partitioned by video and ordered by decided_at descending. Came out to 20000 rows, exactly one per video, as expected.

Listing 15: v_creator_tier_current, for Growth

```
1 DROP VIEW IF EXISTS v_creator_tier_current;
2 CREATE VIEW v_creator_tier_current AS
3 SELECT creator_id, tier, valid_from
4 FROM (
5     SELECT
6         creator_id, tier, valid_from,
7         ROW_NUMBER() OVER (
8             PARTITION BY creator_id ORDER BY valid_from DESC
9         ) AS rn
10    FROM CreatorTierPeriod
11 )
12 WHERE rn = 1;
```

Same latest-row pattern applied against CreatorTierPeriod. Returned 750 rows, one per creator.

Listing 16: v_video_daily_engagement, for the analytics team

```
1 DROP VIEW IF EXISTS v_video_daily_engagement;
```

```

2 CREATE VIEW v_video_daily_engagement AS
3 WITH impression_days AS (
4     SELECT video_id, date(ts) AS day, COUNT(*) AS impressions
5     FROM Impression
6     GROUP BY video_id, date(ts)
7 ),
8 view_days AS (
9     SELECT i.video_id, date(i.ts) AS day,
10         COUNT(*) AS views,
11         SUM((julianday(vs.ended_at) - julianday(vs.started_at)) * 86400.0)
12         AS watch_seconds
13     FROM ViewSegment vs
14     JOIN Impression i ON i.impression_id = vs.impression_id
15     GROUP BY i.video_id, date(i.ts)
16 ),
17 like_days AS (
18     SELECT video_id, date(liked_at) AS day, COUNT(*) AS like_count
19     FROM Like_
20     GROUP BY video_id, date(liked_at)
21 ),
22 retraction_days AS (
23     SELECT l.video_id, date(r.retracted_at) AS day, COUNT(*) AS retraction_count
24     FROM Retraction r
25     JOIN Like_ l ON l.like_id = r.like_id
26     GROUP BY l.video_id, date(r.retracted_at)
27 )
28 SELECT
29     id.video_id, id.day, id.impressions,
30     COALESCE(vd.views, 0) AS views,
31     COALESCE(vd.watch_seconds, 0) AS watch_seconds,
32     COALESCE(ld.like_count, 0) - COALESCE(rd.retraction_count, 0) AS net_likes
33 FROM impression_days id
34 LEFT JOIN view_days vd ON vd.video_id = id.video_id AND vd.day = id.day
35 LEFT JOIN like_days ld ON ld.video_id = id.video_id AND ld.day = id.day
36 LEFT JOIN retraction_days rd ON rd.video_id = id.video_id AND rd.day = id.day;

```

This pulls together impressions, views, watch seconds, and net likes per video per day, built from four grouped subqueries stitched together with LEFT JOINS so that a day with an impression but no engagement still shows zeros instead of simply not appearing. This was checked directly: one day for video 1 shows an impression but zero views and zero net likes, confirming the zero day requirement actually holds rather than just being asserted. Row count came out to 123765.

Listing 17: v_turn_cost, for Finance

```

1 DROP VIEW IF EXISTS v_turn_cost;
2 CREATE VIEW v_turn_cost AS
3 SELECT
4     t.turn_id, t.session_id, t.occurred_at,
5     tu.input_tokens, tu.output_tokens, tu.cached_tokens,
6     mp.input_rate, mp.output_rate, mp.cached_input_rate,
7     (tu.input_tokens * mp.input_rate
8      + tu.output_tokens * mp.output_rate
9      + tu.cached_tokens * mp.cached_input_rate) AS turn_cost
10 FROM Turn t
11 JOIN TurnUsage tu ON tu.turn_id = t.turn_id
12 JOIN ModelPricePeriod mp
13     ON mp.model_id = t.model_id AND mp.priced_at = t.priced_at;

```

This joins each turn to its token usage and to whatever price was active at that turn's own timestamp, using the turn's own priced_at column against ModelPricePeriod's own key rather than pulling whatever the current price happens to be. Row count came out to 7004, matching every turn in the table.

Updatability

An insert was tried straight through v_public_profile against the live database, and it failed immediately with the error that v_public_profile cannot be modified because it is a view.

SQLite views never allow writes at all, no matter how simple the underlying query looks. Postgres works differently here: it can let a write pass through a view automatically when the view's output maps back to exactly one row in exactly one base table, with no aggregation, no DISTINCT, and no set operators, since a write then has one clear target row to land on. `v_public_profile` would actually qualify for that in Postgres, since it is one row per user with nothing aggregated. `v_video_daily_engagement` would never qualify anywhere though, since it pulls from four different tables grouped by day, and there is no single row in any single base table that a write could possibly map back to. So the thing that really decides updatability is not a feature some engine happens to support, it is whether the output traces back to one identifiable row, and SQLite simply chooses not to support even the cases where that would technically be possible.

The As-Of Problem

`v_creator_tier_current` only ever shows tier as of right now, whenever the view happens to be queried, because a view cannot take an input parameter. Its defining query is fixed text that runs fresh every time with no way to hand it a timestamp.

To get tier as of some past date `T`, something that can actually take `T` as an input is needed instead, a parameterised query run directly against `CreatorTierPeriod` checking `valid_from` less than or equal to `T` and `valid_to` being either null or after `T`. A view alone cannot do this, since it is defined once with no runtime arguments, and can only ever represent one fixed moment.

7.2 Transactions

Three scripts were built and run against the live database, each one demonstrating a specific failure mode from section 2.7 of the brief, with a genuine injected failure and a real check afterward proving things stayed consistent.

Listing 18: T1, retraction atomicity, corrected working version

```

1 target_like = conn.execute(
2     "SELECT like_id FROM Like_ WHERE retracted_at IS NULL LIMIT 1"
3 ).fetchone()[0]
4
5 try:
6     conn.execute("BEGIN")
7     conn.execute(
8         "UPDATE Like_ SET retracted_at = '2026-09-01T00:00:00Z' WHERE like_id = ?",
9         (target_like,)
10    )
11    conn.execute(
12        "INSERT INTO Retraction (like_id, retracted_at) VALUES (?, '2026-09-01T00:00:00Z')",
13        (target_like,)
14    )
15    conn.execute(
16        "INSERT INTO Retraction (retraction_id, like_id, retracted_at) VALUES (1, ?,
17        '2026-09-01T00:00:00Z')",
18        (target_like,)
19    )
20    conn.execute(
21        "INSERT INTO Retraction (retraction_id, like_id, retracted_at) VALUES (1, ?,
22        '2026-09-01T00:00:00Z')",
23        (target_like,)
24    )
25    conn.commit()
26 except Exception as e:
27     conn.rollback()
28     print("Transaction failed and was rolled back ->", e)

```

Updated `Like_.retracted_at` and inserted a new `Retraction` row, then deliberately triggered a genuine failure by reusing the same `retraction_id` in a second insert, which SQLite rejects with a real unique constraint error. The first attempt at this used `SELECT 1/0` as the injected failure, expecting an error, which did not actually happen, since SQLite's integer division by zero just returns NULL rather than raising an exception, so the transaction silently committed both writes, which was the opposite of the intended demonstration. The corrected version above forces a genuine `sqlite3.IntegrityError` with a duplicate primary key insert instead, and after that fix both

writes rolled back cleanly, `retracted_at` stayed unchanged and no extra retraction row appeared, confirming the writes really are atomic together. The script's own final printed check read `Consistent: True`.

Listing 19: T2, moderation decision visibility

```
1 BEGIN;
2     INSERT INTO VideoModerationEvent (video_id, state, decided_by_type, decided_by_id,
3         decided_at)
4     VALUES (1, 'taken_down', 'reviewer', NULL, '2026-09-01T00:00:00Z');
5     -- deliberately left open, not committed yet
```

One connection opened a transaction, inserted a new `taken_down` event, and left it uncommitted. A second connection querying at the same time under WAL mode saw a row count of zero for that event, confirming the half applied decision stays genuinely invisible until it is actually committed, not just blocked from being written in the first place.

Listing 20: T3, handle change limit and concurrent write behaviour

```
1 BEGIN;
2     UPDATE Handle SET ended_at = '2026-09-01T00:00:00Z'
3     WHERE user_id = 20 AND ended_at IS NULL;
4     INSERT INTO Handle (user_id, handle, started_at, ended_at)
5     VALUES (20, 'newhandle1', '2026-09-01T00:00:00Z', NULL);
6 COMMIT;
7
8 BEGIN;
9     UPDATE Handle SET ended_at = '2026-09-02T00:00:00Z'
10    WHERE user_id = 20 AND ended_at IS NULL;
11    INSERT INTO Handle (user_id, handle, started_at, ended_at)
12    VALUES (20, 'newhandle2', '2026-09-02T00:00:00Z', NULL);
13 COMMIT;
14
15 BEGIN;
16    UPDATE Handle SET ended_at = '2026-09-03T00:00:00Z'
17    WHERE user_id = 20 AND ended_at IS NULL;
18    INSERT INTO Handle (user_id, handle, started_at, ended_at)
19    VALUES (20, 'newhandle3', '2026-09-03T00:00:00Z', NULL);
20 COMMIT;
```

Three handle changes were tried for the same user. The first two went through fine, and the third was rejected by application logic before it could even reach the database, since SQLite has no declarative way to express a twice-a-year limit on its own, matching what was already written down in the E.2 constraint trace table. Separately, a second connection trying to write while the first held an open transaction got the exact error database is locked, SQLite's version of `SQLITE_BUSY`. This shows SQLite only ever allows one writer at a time, which sidesteps a whole category of concurrent write conflict that a real multi writer engine has to solve with row level locking, at the cost of the second writer simply having to wait or retry instead of writing at the same time.

8 Bonus

Trigger-Based Audit Log for Moderation Decisions

Listing 21: Bonus 1, audit log trigger

```
1 CREATE TABLE IF NOT EXISTS ModerationAuditLog (  
2     audit_id INTEGER PRIMARY KEY,  
3     event_id INTEGER NOT NULL,  
4     video_id INTEGER NOT NULL,  
5     logged_at TEXT NOT NULL  
6 );  
7  
8 DROP TRIGGER IF EXISTS trg_moderation_audit;  
9 CREATE TRIGGER trg_moderation_audit  
10 AFTER INSERT ON VideoModerationEvent  
11 FOR EACH ROW  
12 BEGIN  
13     INSERT INTO ModerationAuditLog (event_id, video_id, logged_at)  
14     VALUES (NEW.event_id, NEW.video_id, NEW.decided_at);  
15 END;
```

A separate ModerationAuditLog table was created, and an AFTER INSERT trigger on VideoModerationEvent writes one row into it every time a new moderation event is recorded. This was tested directly: before a test insert the audit log held 0 rows, after inserting one new moderation event it held exactly 1 row, confirming the trigger fires correctly and automatically on every insert with no application code needed to remember the audit write separately. SQLite only supports FOR EACH ROW triggers, meaning the logic runs once per affected row rather than once per statement, which is exactly what this design assumes.

Trigger Preventing Overlapping Validity Intervals

Listing 22: Bonus 2, overlap prevention trigger

```
1 DROP TRIGGER IF EXISTS trg_prevent_overlapping_tier_periods;  
2 CREATE TRIGGER trg_prevent_overlapping_tier_periods  
3 BEFORE INSERT ON CreatorTierPeriod  
4 FOR EACH ROW  
5 WHEN EXISTS (  
6     SELECT 1 FROM CreatorTierPeriod  
7     WHERE creator_id = NEW.creator_id  
8         AND valid_from < COALESCE(NEW.valid_to, '9999-12-31')  
9         AND COALESCE(valid_to, '9999-12-31') > NEW.valid_from  
10 )  
11 BEGIN  
12     SELECT RAISE(ABORT, 'overlapping tier period for this creator');  
13 END;
```

A BEFORE INSERT trigger on CreatorTierPeriod checks, for the same creator, whether the new row's valid_from to valid_to range overlaps any existing period already stored for that creator, and aborts the insert if it does. This was tested directly, an existing tier period for creator 253 was deliberately reinserted with the same dates, and the trigger correctly rejected it with an overlap error.

What this version cannot guarantee that a declarative constraint could: Postgres EXCLUDE constraints enforce this rule at the database level for every write path including UPDATE, and are checked atomically as part of the same constraint system that backs primary and foreign keys. This trigger only fires on INSERT, so a later UPDATE that shifts an existing period's dates into overlap with another one would not be caught unless a second, separate trigger were written for UPDATE as well. It also does not close a race condition: two separate connections could each run the overlap check, each see no conflict, and then both commit an overlapping insert, since the check and the insert are not part of one atomic operation the way a real constraint would be.

Enforcing the Deletion Grace Period

There is no row level security available in SQLite the way Postgres offers it, so this rule cannot be enforced at the database engine level for every possible query. Enforcement instead lives entirely in the view layer: v_public_profile

filters explicitly on `account.state` equal to `active`, which excludes both deactivated accounts and accounts inside their 30 day deletion window, and this was the intended access path for the mobile client named in section 2.6 of the brief.

What defeats this: any query written directly against the `AppUser` table instead of through `v_public_profile` bypasses the filter completely, since the base table itself has no such restriction built in. This is a real and honestly stated limitation, not a flaw hidden by the design, since SQLite gives no mechanism to force every query, present or future, through a single filtered access path the way Postgres row level security would. The enforcement here depends on discipline, that anyone querying user data goes through the view rather than the table, which is weaker than a database level guarantee but is the best available option without switching engines.